ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 7

C++ • Class, Constructor, Destructor • 18 տարբերակ • Ֆայլից տվյալների ընթերցում

Ընդհանուր պահանջներ

Յուրաքանչյուր առաջադրանքի համար գրել ամբողջական C++ ծրագիր՝ main() ֆունկցիայով: Դասի դաշտերը հայտարարել private, մեթոդները՝ public: Կիրառել առնվազն երկու կոնստրուկտոր (լռելյայն և պարամետրերով), սկզբնարժեքավորման ցուցակ և դեստրուկտոր:

Մուտքային տվյալները կարդալ input.txt տեքստային ֆայլից՝ օգտագործելով <fstream> և ifstream: Մշակման բոլոր արդյունքները, հաշվարկները, սխալների մասին հաղորդագրությունները և օբյեկտների ստեղծման/ոչնչացման գրանցումները գրել առանձին output.txt ֆայլում՝ օգտագործելով ofstream: Ծրագրի արդյունքները չարտածել էկրանին (cout):

Դեստրուկտորի գրանցման համար կարելի է դասին փոխանցել ofstream& հղում և պահել այն որպես անդամ: Այդ հղումը պետք է վերաբերի դեռ բաց ֆայլին: Օբյեկտները ստեղծել ելքային հոսքից հետո և ոչնչացնել մինչև հոսքի փակվելը:

Պարտադիր է ստուգել input.txt և output.txt ֆայլերի բացվելը, N-ի դրական լինելը, գրառումների ճիշտ ընթերցումը և սխալ տվյալները: Յուրաքանչյուր գրառման համար ստեղծել համապատասխան դասի օբյեկտ, կատարել պահանջվող գործողությունները և արդյունքները գրել output.txt-ում: Դեստրուկտորի հաղորդագրությունները նույնպես գրանցել output.txt-ում՝ ապահովելով, որ ելքային ֆայլը բաց մնա օբյեկտների ոչնչացման պահին:

input.txt-ից ընթերցման և output.txt-ում գրանցման ընդհանուր օրինակ

```
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream input("input.txt");
    ofstream output("output.txt");
    if (!output.is_open()) return 1;
    if (!input.is_open()) {
        output << "Cannot open input.txt\n";
        return 1;
    }
    int n;
    if (!(input >> n) || n <= 0) {
        output << "Invalid record count\n";
```

```
        return 1;
    }
    for (int i = 0; i < n; ++i) {
        // Կարդալ առաջադրանքի դաշտերը input-ից:
        // Ստեղծել օբյեկտ և կատարել գործողությունները:
        // Արդյունքները գրել output հոսքում:
    }
    output << "Processing finished\n";
    return 0;
}
```

Առաջադրանք N1. Գրադարանի գրքերի հաշվառում

Դաս՝ Book Դաշտեր՝ title, author, pages, availableCopies.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել առանց պարամետրերի և պարամետրերով կոնստրուկտորներ: Սխալ էջաքանակը փոխարինել 1-ով, իսկ օրինակների բացասական քանակը՝ 0-ով:

Մեթոդներ և դեստրուկտոր: Իրականացնել borrowBook() և returnBook() մեթոդները՝ կանխելով բացասական մնացորդը: Դեստրուկտորում output.txt-ում գրանցել գրքի անվանումը և հաշվառումից դուրս գալու հաղորդագրություն:

Ստուգում main()-ում: main()-ում ստեղծել առնվազն երեք գիրք տարբեր կոնստրուկտորներով և բոլոր գործողությունների արդյունքները գրել output.txt-ում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ title author pages availableCopies.

Ֆայլի օրինակ (2 գրառում).

```
2
Clean_Code Robert_Martin 450 5
Clean_Code Robert_Martin 450 5
```

Լրացուցիչ պահանջ. Գրքերի քանակը նվազեցնել/ավելացնել borrowBook() և returnBook() մեթոդներով: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N2. Հյուրանոցի սենյակների ամրագրում

Դաս՝ HotelRoom Դաշտեր՝ number, nightlyRate, occupied.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել լռելյայն, երկպարամետր և եռապարամետր կոնստրուկտորներ: Գնի բացասական արժեքը չընդունել:

Մեթոդներ և դեստրուկտոր: Ավելացնել checkIn(), checkOut(), calculateCost(days): Արգելել զբաղված սենյակի կրկնակի ամրագրումը: Դեստրուկտորում output.txt-ում գրանցել սենյակի ազատման մասին:

Ստուգում main()-ում: Ստեղծել երեք սենյակ, կատարել հաջող և անհաջող ամրագրումներ, հաշվել 4 գիշերվա արժեքը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ number nightlyRate occupied.

Ֆայլի օրինակ (2 գրառում).

```
2
101 25000 0
101 25000 0
```

Լրացուցիչ պահանջ. occupied դաշտի համար օգտագործել 0 կամ 1: Փորձարկել checkIn(), checkOut(), calculateCost(4): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N3. Առցանց խանութի զամբյուղ

Դաս՝ Cart Դաշտեր՝ customer, total, itemCount.

Կոնստրուկտորներ և սկզբնական վիճակ: Իրականացնել կոնստրուկտորների գերբեռնում՝ դատարկ և նախապես լրացված զամբյուղների համար:

Մեթոդներ և դեստրուկտոր: addItem(price), removeItem(price), checkout() մեթոդներով վերահսկել գումարն ու ապրանքների քանակը: Դեստրուկտորը ցույց տա վերջնական վիճակը:

Ստուգում main()-ում: Ցուցադրել երկու հաճախորդի զամբյուղներ և ստուգել սխալ գներով գործողությունները: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ customer total itemCount.

Ֆայլի օրինակ (2 գրառում).

```
2
Anna 15000 3
Anna 15000 3
```

Լրացուցիչ պահանջ. Սկզբնական զամբյուղի վրա կատարել addItem(), removeItem(), checkout(): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N4. Խաղի հերոսի էներգիայի կառավարում

Դաս՝ GameCharacter Դաշտեր՝ nickname, health, energy.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել երեք կոնստրուկտոր. լռելյայն, միայն անունով և ամբողջական: Առողջությունը սահմանափակել 0–100 միջակայքում:

Մեթոդներ և դեստրուկտոր: Իրականացնել attack(cost), heal(points), rest(): Հետևել, որ էներգիան բացասական չդառնա: Դեստրուկտորում output.txt-ում գրանցել հերոսի հեռանալու հաղորդագրություն:

Ստուգում main()-ում: Ներքին բլոկում ստեղծել երկրորդ հերոս և հետևել դեստրուկտորների կանչի հերթականությանը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ nickname health energy.

Ֆայլի օրինակ (2 գրառում).

```
2
Archer 85 60
Archer 85 60
```

Լրացուցիչ պահանջ. Կատարել attack(20), heal(15), rest() և ստուգել սահմանները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N5. Ուսանողի դասընթացի առաջադիմություն

Դաս՝ StudentCourse Դաշտեր՝ studentName, courseName, gradesCount, gradesSum.

Կոնստրուկտորներ և սկզբնական վիճակ: Գրել երկու պարամետրով և լռելյայն կոնստրուկտորներ՝ սկզբնարժեքավորման ցուցակով:

Մեթոդներ և դեստրուկտոր: addGrade(grade) ընդունի միայն 0–20 գնահատականները, average() հաշվարկի միջինը, status() որոշի՝ միջինը բավարար է արդյոք (>=10): Դեստրուկտորում ցույց տալ արդյունքը:

Ստուգում main()-ում: Ստեղծել մի քանի օբյեկտ, ստուգել նաև գնահատական չունեցող ուսանողի դեպքը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ studentName courseName gradesCount gradesSum.

Ֆայլի օրինակ (2 գրառում).

```
2
Ani Programming 3 48
Ani Programming 3 48
```

Լրացուցիչ պահանջ. gradesSum-ը նախապես ստացված գնահատականների գումարն է: Ավելացնել մեկ գնահատական և հաշվել միջինը: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N6. Ավտովարձույթի պայմանագիր

Դաս՝ RentalCar Դաշտեր՝ plate, dailyPrice, rentedDays.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել պարամետրերով կոնստրուկտոր և լռելյայն տարբերակ: Վավերացնել գինն ու օրերի քանակը:

Մեթոդներ և դեստրուկտոր: extendRental(days), totalPrice(), closeRental(): Փակված պայմանագիրը այլևս չերկարաձգել: Դեստրուկտորը տեղեկացնի պայմանագրի ավարտի մասին:

Ստուգում main()-ում: Ստեղծել երկու պայմանագիր, փորձարկել երկարաձգում և կրկնակի փակում: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ plate dailyPrice rentedDays.

Ֆայլի օրինակ (2 գրառում).

```
2
AA123BB 12000 3
AA123BB 12000 3
```

Լրացուցիչ պահանջ. Կատարել extendRental(2), totalPrice(), closeRental(): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N7. Սրճարանի պատվերների համակարգ

Դաս՝ CafeOrder Դաշտեր՝ orderId, tableNumber, totalPrice, isPaid.

Կոնստրուկտորներ և սկզբնական վիճակ: Գրել երկու տարբեր կոնստրուկտոր՝ դատարկ և նախապես որոշված պատվերի համար:

Մեթոդներ և դեստրուկտոր: addDish(price), pay(amount), remainingPayment(): Արգելել արդեն վճարված պատվերի փոփոխությունը: Դեստրուկտորում output.txt-ում գրանցել պատվերի ամփոփումը:

Ստուգում main()-ում: Փորձարկել ճիշտ վճարում, անբավարար վճարում և բացասական գին: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ orderId tableNumber totalPrice isPaid.

Ֆայլի օրինակ (2 գրառում).

```
2
1001 5 8200 0
1001 5 8200 0
```

Լրացուցիչ պահանջ. isPaid-ը 0 կամ 1 է: Կատարել addDish(), pay(), remainingPayment(): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N8. Բանկային հաշիվ՝ գործարքների պատմությամբ

Դաս՝ BankAccount Դաշտեր՝ accountNumber, owner, balance, transactionCount.

Կոնստրուկտորներ և սկզբնական վիճակ: Օգտագործել const accountNumber անդամ և սկզբնարժեքավորման ցուցակ: Ստեղծել գերբեռնված կոնստրուկտորներ:

Մեթոդներ և դեստրուկտոր: deposit(amount), withdraw(amount), showSummary(): Հաշվել հաջող գործարքները, կանխել ոչ բավարար մնացորդով ելքերը: Դեստրուկտորը արտածի հաշվի փակման տեղեկություն:

Ստուգում main()-ում: Ստեղծել հաշիվներ տարբեր սկզբնական մնացորդներով և ստուգել սահմանային դեպքերը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ accountNumber owner balance transactionCount.

Ֆայլի օրինակ (2 գրառում).

```
2
101 Ani 50000 2
101 Ani 50000 2
```

Լրացուցիչ պահանջ. Կատարել deposit(), withdraw() և արտածել գործարքների քանակը: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N9. Կինոթատրոնի տոմսերի կառավարում

Դաս՝ MovieSession Դաշտեր՝ movieTitle, capacity, soldTickets.

Կոնստրուկտորներ և սկզբնական վիճակ: Իրականացնել կոնստրուկտորների գերբեռնում և ապահովել, որ վաճառված տոմսերը չգերազանցեն տեղերը:

Մեթոդներ և դեստրուկտոր: sellTickets(count), refundTickets(count), freeSeats(): Դեստրուկտորում output.txt-ում գրանցել նիստի վերջնական վիճակը:

Ստուգում main()-ում: Փորձարկել 0, բացասական և չափազանց մեծ քանակներով հարցումներ: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ movieTitle capacity soldTickets.

Ֆայլի օրինակ (2 գրառում).

```
2
Interstellar 80 25
Interstellar 80 25
```

Լրացուցիչ պահանջ. Կատարել sellTickets(), refundTickets(), freeSeats(): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N10. Առաքման ծանրոցների հետևում

Դաս՝ Parcel Դաշտեր՝ trackingCode, weight, delivered.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել լռելյայն և պարամետրերով կոնստրուկտորներ: Քաշը պետք է լինի դրական:

Մեթոդներ և դեստրուկտոր: shippingCost() հաշվարկի արժեքը՝ մինչև 2 կգ՝ 1000 դրամ, դրանից բարձր յուրաքանչյուր սկսված կգ-ի համար +400 դրամ: markDelivered() փոխի կարգավիճակը միայն մեկ անգամ: Դեստրուկտորում տպել վիճակը:

Ստուգում main()-ում: Ստուգել 1.5, 2 և 3.2 կգ ծանրոցների արժեքները: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ trackingCode weight delivered.

Ֆայլի օրինակ (2 գրառում).

```
2
PK100 3.2 0
PK100 3.2 0
```

Լրացուցիչ պահանջ. Հաշվել shippingCost() և կատարել markDelivered(): Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N11. Սպորտային ակումբի անդամակցություն

Դաս՝ Membership Դաշտեր՝ memberName, months, monthlyFee, active.

Կոնստրուկտորներ և սկզբնական վիճակ: Գրել երեք կոնստրուկտոր՝ տարբեր սկզբնական տվյալներով:

Մեթոդներ և դեստրուկտոր: renew(extraMonths), cancel(), totalFee(): Չեղարկված անդամակցությունը չերկարաձգել առանց նոր ակտիվացման մեթոդի: Դեստրուկտորը գրանցի անդամակցության ավարտը:

Ստուգում main()-ում: Փորձարկել 1, 6 և 12 ամսվա անդամակցություններ: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ memberName, months, monthlyFee, active.

Ֆայլի օրինակ (2 գրառում).

```
2
Sample 5 5 0
Sample 5 5 0
```

Լրացուցիչ պահանջ. Ընթերցված դաշտերի քանակը համապատասխանեցնել առաջադրանքի դասին: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում

յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N12. Պահեստի ապրանքների պաշար

Դաս՝ WarehouseItem Դաշտեր՝ sku, name, quantity, unitPrice.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել լռելյայն և պարամետրերով կոնստրուկտորներ՝ ստուգելով բոլոր թվային դաշտերը:

Մեթոդներ և դեստրուկտոր: receive(amount), ship(amount), inventoryValue(): Չթույլատրել բացասական մնացորդ: Դեստրուկտորը ցույց տա վերջնական արժեքը:

Ստուգում main()-ում: Ստեղծել երեք ապրանք և կատարել հաջորդական մուտքեր ու ելքեր: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ sku, name, quantity, unitPrice.

Ֆայլի օրինակ (2 գրառում).

```
2
5 Sample 5 5
5 Sample 5 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N13. Համակարգչային դասարանի սարքավորումներ

Դաս՝ Computer Դաշտեր՝ inventoryId, model, poweredOn, usageMinutes.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել գերբեռնված կոնստրուկտորներ՝ տարբեր մոդելներով համակարգիչներ ստեղծելու համար:

Մեթոդներ և դեստրուկտոր: powerOn(), use(minutes), powerOff(): Աշխատաժամանակը ավելացնել միայն միացված համակարգչի դեպքում: Դեստրուկտորը տպի ընդհանուր օգտագործման ժամանակը:

Ստուգում main()-ում: Ներքին բլոկում ստեղծել ժամանակավոր համակարգիչ և դիտարկել ոչնչացման պահը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ inventoryId, model, poweredOn, usageMinutes.

Ֆայլի օրինակ (2 գրառում).

```
2
Sample Sample 5 5
Sample Sample 5 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N14. Ավտոկայանատեղիի հաշվառում

Դաս՝ ParkingTicket Դաշտեր՝ carNumber, entryHour, exitHour, closed.

Կոնստրուկտորներ և սկզբնական վիճակ: Պարամետրերով կոնստրուկտորով ընդունել մեքենայի համարն ու մուտքի ժամը (0–23):

Մեթոդներ և դեստրուկտոր: closeTicket(exitHour), calculateFee(): Յուրաքանչյուր սկսված ժամի արժեքը 300 դրամ է. եթե ելքի ժամը փոքր է մուտքի ժամից, համարել հաջորդ օր: Կրկնակի փակումն արգելել: Դեստրուկտորում ցույց տալ հաշիվը:

Ստուգում main()-ում: Փորձարկել նույն օրվա և կեսգիշերից անցնող կայանումը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ carNumber, entryHour, exitHour, closed.

Ֆայլի օրինակ (2 գրառում).

```
2
5 5 5 0
5 5 5 0
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N15. Դասընթացի տեղերի բաշխում

Դաս՝ TrainingCourse Դաշտեր՝ title, capacity, enrolled.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել լռելյայն, երկպարամետր և եռապարամետր կոնստրուկտորներ՝ պատվիրակող կոնստրուկտորի կիրառմամբ:

Մեթոդներ և դեստրուկտոր: enroll(count), cancelEnrollment(count), freePlaces(): Վավերացնել քանակները: Դեստրուկտորը տպի զբաղեցված և ազատ տեղերը:

Ստուգում main()-ում: Փորձարկել լրիվ լցված խումբ և տեղերի գերազանցման փորձ: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ title, capacity, enrolled.

Ֆայլի օրինակ (2 գրառում).

```
2
Sample 5 5
Sample 5 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N16. Սենսորի չափումների հավաքագրում

Դաս՝ TemperatureSensor Դաշտեր՝ sensorId, lastValue, minValue, maxValue, measurements.

Կոնստրուկտորներ և սկզբնական վիճակ: Ստեղծել կոնստրուկտորներ՝ առանց չափումների և առաջին չափումով:

Մեթոդներ և դեստրուկտոր: record(value) ընդունի -50-ից +100°C արժեքներ, թարմացնի նվազագույնը, առավելագույնը և քանակը: showStats() ճիշտ աշխատի նաև դատարկ վիճակում: Դեստրուկտորը ամփոփի չափումները:

Ստուգում main()-ում: Ստեղծել երկու սենսոր և համեմատել նրանց վիճակները: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ sensorId, lastValue, minValue, maxValue, measurements.

Ֆայլի օրինակ (2 գրառում).

```
2
Sample 5 5 5 5
Sample 5 5 5 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N17. Դինամիկ զանգվածի կառավարում

Դաս՝ IntBuffer Դաշտեր՝ int* data, int size.

Կոնստրուկտորներ և սկզբնական վիճակ: explicit կոնստրուկտորով ընդունել չափը և new[]-ով հատկացնել հիշողություն: Ձրոյացնել տարրերը:

Մեթոդներ և դեստրուկտոր: Գրել set(index,value), get(index), sum() և ~IntBuffer()՝ delete[] կիրառմամբ: Պատճենումը կանխել՝ ջնջելով պատճենման կոնստրուկտորն ու վերագրման օպերատորը, կամ իրականացնել խորքային պատճենում:

Ստուգում main()-ում: Ստեղծել զանգվածը ներքին բլոկում, ցուցադրել տարրերը և հետևել ազատմանը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ int* data, int size.

Ֆայլի օրինակ (2 գրառում).

```
2
5 5
5 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը:

Առաջադրանք N18. Ռեսուրսների հերթական ազատում

Դաս՝ EventLogger Դաշտեր՝ eventName, sequenceNumber.

Կոնստրուկտորներ և սկզբնական վիճակ: Կոնստրուկտորը տպի ստեղծման հերթական համարը, դեստրուկտորը՝ ավարտման մասին հաղորդագրություն:

Մեթոդներ և դեստրուկտոր: Ստեղծել արտաքին բլոկում մեկ, ներքին բլոկում երկու օբյեկտ: Ավելացնել ֆունկցիա, որը ստեղծում է ժամանակավոր օբյեկտ: Համեմատել ստեղծման և ոչնչացման հերթականությունը:

Ստուգում main()-ում: Գրել սպասվող արտածումը մինչև ծրագրի գործարկումը և բացատրել հակառակ հերթականությունը: Բոլոր ստացված արդյունքները գրանցել output.txt ֆայլում, ոչ թե կոնսոլում:

Ֆայլային մուտք՝ input.txt | Ֆայլային ելք՝ output.txt

Դաշտերի հերթականությունը՝ eventName, sequenceNumber.

Ֆայլի օրինակ (2 գրառում).

```
2
Sample 5
Sample 5
```

Լրացուցիչ պահանջ. Օգտագործել առաջադրանքի նկարագրած մեթոդները: Առնվազն մեկ սխալ գրառում փորձարկել առանձին ֆայլով և բացատրել ծրագրի արձագանքը:

Նշում. Օրինակի թվերը ուսանողը կարող է փոխել, սակայն դաշտերի հերթականությունը և տիպերը պետք է համապատասխանեն դասի կառուցվածքին: output.txt-ում յուրաքանչյուր գրառման համար նշել գրառման համարը, մշակման արդյունքը և անհրաժեշտության դեպքում սխալի պատճառը: